

Module 06: Inheritance and Polymorphism

Inheritance is the feature Java was sold on in 1995 and the one modern Java advises you to use sparingly. Both of those are true, and this module covers why.

The part you will use every single day is not `extends` at all. It is the `equals` / `hashCode` contract, and this module shows it breaking rather than describing it.

What You'll Learn

- Dynamic dispatch, and why it is the whole point of inheritance
- `super`, and why `@Override` is worth writing every time
- `abstract` and `final` as opposite instructions about your design
- The `equals` / `hashCode` contract, demonstrated by breaking a `HashSet`
- What is *not* polymorphic: fields and static methods

Coming From Another Language

	Python	Java
Inherit	<code>class Dog(Animal)</code>	<code>class Dog extends Animal</code>
Call parent	<code>super().__init__()</code>	<code>super(...)</code> , and it must come first
Multiple inheritance	yes	no, single only. Interfaces fill the gap.
Abstract	<code>abc.ABC</code>	<code>abstract</code> keyword
Preventing subclassing	not really	<code>final</code>
Equality	<code>__eq__</code> and <code>__hash__</code>	<code>equals</code> and <code>hashCode</code> , same pairing

Java allows only one parent class. That restriction is why interfaces exist, and Module 07 covers them.

The Lesson

Dynamic dispatch

One call site, three methods. From `Inheritance.java`:

```
[log] system reboot
[email to ada@example.com] system reboot
[sms to +44 7700 900000] system reboot
```

The loop is written against `Notification`. The compiler checks only that the type *has* a `deliver()` method. Which one runs is decided at runtime from the actual object.

The practical payoff: you can add a fourth notification type without touching that loop. Code written against the base type keeps working.

super and @Override

`super(...)` calls the parent constructor and must come first, with the exception Module 05 covered.

`super.method()` calls the parent's version, which is how you extend behaviour rather than replace it.

Always write `@Override`. It is not decoration, it is an instruction to the compiler to check that you really are overriding something. `OverrideTypo.java` misspells `deliver` as `delivar`:

```
error: delivar() in Email does not override or implement a method from a supertype
    @Override
    ^
```

Without the annotation that compiles perfectly, adds a method nobody calls, and leaves the base implementation running. The code looks right, so the bug survives review. One annotation removes the entire category.

abstract and final

`abstract` says subclasses must fill this in. `final` says nobody may change this. From `AbstractAndFinal.java`:

```
abstract class Shape {
    abstract double area();

    final boolean isLargerThan(Shape other) {
        return this.area() > other.area();
    }
}
```

An abstract class cannot be constructed, which is the point: a shape with no particular shape has no sensible area. Unlike an interface, it can also hold ordinary methods and state.

`final` on a method blocks overriding. `final` on a class blocks extension entirely. `String` and `Integer` are both final, and not out of unhelpfulness: if anyone could subclass `String` and override `equals`, no code anywhere could trust a `String` again.

Make classes final more often than feels natural. A class not designed for extension usually cannot be extended safely, because a subclass can override any method and break invariants the class relies on. Module 09 introduces `sealed`, the middle ground between all and nothing.

The equals/hashCode contract

This is the part you will actually hit. From `EqualsHashCodeContract.java`, a class that overrides `equals` and forgets `hashCode`:

```
two equal points added, set size: 2
contains an equal point:           false
but the two ARE equal:            true
```

Read that again. The set holds two items it agrees are equal, and cannot find an item equal to one it already contains.

`HashSet` does not call `equals` on everything. It computes `hashCode` to choose a bucket and compares only within that bucket. `Object.hashCode` is identity-based, so two distinct instances get different codes regardless of what `equals` says, land in different buckets, and never meet.

Add `hashCode` and it behaves:

```
two equal points added, set size: 1
contains an equal point:           true
```

The full contract:

1. If `a.equals(b)` then `a.hashCode() == b.hashCode()`. This is the one that breaks things.
2. Equal hash codes do not imply equality. Collisions are legal.
3. Reflexive: `a.equals(a)`
4. Symmetric: `a.equals(b) == b.equals(a)`
5. Transitive: `a=b` and `b=c` implies `a=c`
6. Consistent: the same answer every time, if nothing changed
7. `a.equals(null)` is always false

Use `Objects.hash(...)` and pass **exactly** the fields `equals` compares. Using a field in one and not the other breaks rule 1 just as thoroughly as omitting `hashCode` altogether.

`toString` is the third of the family. Not part of the contract, but the difference between a readable log line and `BadPoint@4034c28c`.

The shortcut: a record writes all three correctly from its components. Module 08 covers them, and for a data-carrying class a record is almost always the better answer.

What is not polymorphic

Methods dispatch on the object. Fields and statics do not. From `WhatIsNotPolymorphic.java`, where the object is a `Child` held in a `Parent` variable:

```
instance method: Child.instanceName
field access:    parent field
as a Child:     child field
static method:  Parent.staticName
```

Both fields exist on the object at once. The subclass did not replace the parent's field, it added a second one that hides it. Static methods are likewise *hidden*, not overridden.

The rule that follows: **never shadow a field, never redeclare a parent's static method**. Both are legal, neither does what it looks like, and there is no good reason to write either.

This is also why Module 04 warned that `static` blocks you later. A static method cannot be overridden, so it cannot take part in any of this.

Run It

```
java modules/06-inheritance-and-polymorphism/examples/Inheritance.java
java modules/06-inheritance-and-polymorphism/examples/AbstractAndFinal.java
java modules/06-inheritance-and-polymorphism/examples/EqualsHashCodeContract.java
java modules/06-inheritance-and-polymorphism/examples/WhatIsNotPolymorphic.java

# Fails on purpose.
java modules/06-inheritance-and-polymorphism/examples/OverrideTypo.java

./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism
```

Common Mistakes

Overriding `equals` without `hashCode`. Your objects will go into a `HashMap` and never come out. This is the most common serious bug in this module.

Including a field in `hashCode` that `equals` ignores, or the reverse. Same breakage, harder to spot.

Using a mutable field in `equals` and `hashCode`. Put the object in a set, change the field, and it is now lost inside its own collection, because it is filed under a bucket its new hash code no longer matches.

Omitting `@Override`. Free bug detection, declined.

Calling an overridable method from a constructor. The subclass override runs before the subclass constructor has initialised its fields, so it sees nulls and zeros. Make such methods `final` or `private`.

Reaching for inheritance to reuse code. `extends` means "is a", not "borrows from". If the subclass cannot honestly be used everywhere the parent can, use composition: hold the other object as a field.

Key Takeaways

- **Dynamic dispatch picks the method from the object**, not the variable's type.
- **Always write `@Override`**. It converts silent bugs into compile errors.
- **`abstract` forces subclasses to fill in. `final` forbids change.** Prefer `final` by default.
- **Override `equals` and `hashCode` together, over the same fields, always.**
- **Never shadow fields or redeclare static methods.** They are resolved by declared type, not by object.
- **Prefer composition to inheritance** unless the "is a" relationship is honest.

Homework

[homework/README.md](#)

Break a `HashSet` on purpose, diagnose it, then fix it. Reference solution in [solutions/06-inheritance-and-polymorphism/](#).